502094 - Software Deployment, Operations And Maintenance

Module 1: DevOps Fundamentals and Environment Setup

Lecturer: MSc. Mai Van Manh

Email: maivanmanh@tdtu.edu.vn

Module Overview

- Welcome to **Module 1: Introduction to DevOps & Course Setup**
- Today we will:
  - Understand what **DevOps** really is (beyond buzzwords)
  - See the **big picture** of modern web application deployment
  - Explore the **course roadmap** and how modules connect
  - Set up **accounts, tools, and environments**
- Goal:
  - By the end, you know **why** DevOps matters and **what** this course expects from you

1. Welcome & Course Orientation

- Who this course is for:
  - Backend-oriented students
  - Future **DevOps / Cloud / Platform / SRE** engineers
- Course philosophy:
  - Less “clicking in UI”, more **automation and code**
  - Learn by **doing**: labs, exercises, small real-world setups
- You will be treated as:
  - Junior engineers preparing for **real production environments**

Instructor Introduction & Expectations

- Lecturer:
  - **MSc. Mai Van Manh**
  - Focus: software deployment, backend systems, DevOps practices
- Expectations:
  - Bring your **laptop** and be ready to **type, break, fix**
  - Complete labs on time
  - Ask questions early, not only before deadlines
- Classroom culture:
  - Respectful discussion
  - You are allowed to say “I don’t know” — but must be willing to learn

What You Will Be Able to Do

- At the end of this course, you should be able to:
  - Deploy a **web application** (Node.js or Spring Boot) to the cloud
  - Use **Docker** and **Kubernetes** to run services reliably
  - Implement **CI/CD pipelines** with GitHub Actions
  - Codify infrastructure with **Terraform**
  - Configure servers with **Ansible**
  - Add **monitoring, logging, and alerts** to your system
- This is exactly what many **junior DevOps / backend roles** require

DevOps-Related Roles in Industry

- **DevOps Engineer**:
  - Builds and maintains CI/CD, infrastructure automation
- **Site Reliability Engineer (SRE)**:
  - Focus on reliability, availability, incident response
- **Cloud Engineer**:
  - Designs and manages cloud infrastructure (AWS, Azure, GCP)
- **Platform Engineer**:
  - Builds internal platforms for developers (Kubernetes, PaaS)
- This course gives you **foundations** for all these roles

2. What Exactly Is DevOps?

- Misconception:
  - DevOps ≠ “the person who installs Jenkins”
- DevOps is:
  - A **culture** and **set of practices** that bring **Dev** and **Ops** together
  - Supported by tools, but not defined by them
- Key idea:
  - **Collaboration, automation, and shared responsibility** across the software lifecycle

Dev vs Ops: Traditional Silos

- Traditional model:
  - Dev team: “We wrote the code, it works on our machine.”
  - Ops team: “We keep servers stable, please stop breaking them.”
- Problems:
  - Slow releases
  - Blame culture
  - Manual, error-prone deployments
- DevOps emerged to:
  - **Break silos**, integrate development & operations

DevOps Timeline (Simplified)

- Pre-DevOps:
  - Manual deployments, FTP to production, long release cycles
- Early automation:
  - Shell scripts, basic CI servers
- DevOps movement:
  - ~2009, DevOpsDays, “Infrastructure as Code”
- Cloud Native era:
  - Containers, Kubernetes, GitOps, microservices
- Today:
  - DevOps practices are **standard** in modern tech companies

Core Goals of DevOps

- Faster and safer delivery:
  - **Shorter release cycles**, smaller changes
- Higher stability:
  - Fewer incidents, faster recovery (**MTTR**)
- Automation:
  - Reduce manual work, enforce consistency
- Feedback and learning:
  - Use **metrics and logs** to make decisions
- Collaboration:
  - Dev + Ops + QA + Security work together

DevOps Principles: CALMS

- **C – Culture**:
  - Trust, collaboration, blameless postmortems
- **A – Automation**:
  - CI/CD, infrastructure as code, automated testing
- **L – Lean**:
  - Small batch sizes, reducing waste
- **M – Measurement**:
  - Metrics, logging, observability
- **S – Sharing**:
  - Knowledge, tools, best practices across teams

Example: DevOps vs Non-DevOps Release

- Non-DevOps:
  - Big-bang release every 6 months
  - Manual copying of files, manual DB scripts
  - Downtime at midnight, everyone stressed
- DevOps:
  - Small changes deployed **daily**
  - Automated pipeline: build → test → deploy
  - Rollback plan, monitoring, and alerts in place
- Question:
  - Which environment would you prefer to work in?

3. The DevOps Lifecycle (High-Level)

- DevOps lifecycle is not linear, but a **continuous loop**:
  - **Plan → Code → Build → Test → Release → Deploy → Operate → Monitor → (Back to Plan)**
- Each stage should:
  - Be **automated where possible**
  - Produce **feedback** for previous stages
- We will map course modules onto this lifecycle

Plan: Requirements & Collaboration

- Activities:
  - Capture requirements as user stories
  - Track work in an issue tracker (GitHub Issues, Jira)
- DevOps perspective:
  - Plan for **observability and deployment** from the beginning
- Example:
  - A story includes not only “feature works” but also:
    - “Add metrics and logs for this feature”

Code: Version Control & Hygiene

- Tools:
  - **Git** + Git hosting (GitHub)
- Practices:
  - Feature branches
  - Meaningful commit messages
  - Code reviews via pull requests
- Simple Git example:

```
git clone git@github.com:yourname/devops-course-project.git
git checkout -b feature/add-login
git add .
git commit -m "Add login endpoint with JWT"
git push origin feature/add-login
```

Build: From Source to Artifacts

- Build step:
  - Turn source code into **artifacts**:
    - Node.js: compiled bundles, Docker images
    - Java: JAR/WAR files
- Automated builds:
  - Triggered on every push or pull request
- Example (Node.js):

```
npm ci
npm run build
```

- Example (Spring Boot):

```
./mvnw clean package
```

Test: Quality Before Release

- Types of tests:
  - Unit tests, integration tests, end-to-end tests
- CI runs:
  - Tests automatically with each change
- Example (Node.js):

```
npm test
```

- DevOps mindset:
  - “If tests are **not** automated, they are often **not** done.”

Release & Deploy: Getting to Users

- **Release**:
  - Decide **which version** is ready to be deployed
  - Tag and package artifacts
- **Deploy**:
  - Push artifacts to environments:
    - Dev, staging, production
- Strategies:
  - Blue-green, rolling, canary (later modules)
- DevOps aims:
  - Deployments should be:
    - **Predictable, reversible, and repeatable**

Operate & Monitor

- Operate:
  - Keep services running in production
  - Handle scaling, failures, backups
- Monitor:
  - Metrics (CPU, latency, errors)
  - Logs (application, system, audit)
  - Traces (for microservices)
- Observability:
  - Allows you to ask:
    - “What is happening **right now** in production?”

4. CI/CD Overview (Preview)

- CI/CD is at the **heart** of DevOps
- Continuous Integration (CI):
  - Integrate changes frequently, test automatically
- Continuous Delivery / Deployment (CD):
  - Automatically prepare and often deploy changes to environments
- In this course, we will:
  - Build real **GitHub Actions** pipelines

4.1 Continuous Integration (CI)

- Definition:
  - Practice of merging code frequently into a shared branch and running **automated build & tests**
- Why:
  - Detect integration issues early
  - Avoid “merge hell”
- Example GitHub Actions trigger:

```yaml
on:
  push:
    branches: [ "main" ]
  pull_request:
    branches: [ "main" ]
```

- CI fails?

4.2 Continuous Delivery (CD)

- CD:
  - Automatically prepares a **deployable build** for any change that passes CI
- Activities:
  - Build artifacts
  - Tag versions
  - Publish to artifact repositories
- Example target:
  - After CI, upload a JAR artifact or push a Docker image
- Production deployment may still need:
  - Manual approval or extra checks

4.3 Continuous Deployment (Preview)

- Continuous Deployment:
  - Every change that passes tests is **automatically deployed** to production
- Requirements:
  - Strong test suite
  - Observability and fast rollback
- Safety:
  - Manual approvals for critical systems
  - Feature flags to control exposure

4.4 CI/CD Pipeline Architecture (High Level)

- Typical flow:
  - **Source code** → Build → Test → Package → Release → Deploy
- Example (Node.js web app):
  - Push to `main`:
    - Run `npm ci`, `npm test`
    - Build Docker image
    - Push to Docker Hub
    - Deploy to EC2 or Kubernetes
- This course:
  - Will walk through **each piece** in later modules

5. Modern Web Application Deployment Architecture

- Goal:
  - Understand the **big picture** before diving into details
- Common environments:
  - Local development
  - Dev / staging
  - Production
- Each environment:
  - Has similar structure but different **scale and risk**

Environments: Local → Dev → Staging → Production

- **Local**:
  - Run app on your laptop
- **Dev**:
  - Shared environment for early integration
- **Staging**:
  - Production-like environment for final testing
- **Production**:
  - Real users, real money, real consequences
- DevOps practice:
  - Reduce differences between environments

Deployment Models

- Single server deployments:
  - Simple, good for small projects
- Containerized deployments (Docker):
  - Package app + dependencies together
- Orchestrated deployments (Kubernetes):
  - Manage many containers across many nodes
- This course:
  - Starts from simple servers → containers → Kubernetes

Key Components in a Web Deployment

- Reverse proxy:
  - **Nginx**, **Caddy** — terminate HTTPS, route requests
- Application layer:
  - Node.js (Express, NestJS)
  - Java (Spring Boot)
- Data layer:
  - Relational DB (PostgreSQL, MySQL)
  - Caching (Redis)
- Edge / load balancer:
  - Cloud load balancers (ALB, NLB)
- Observability:
  - Prometheus, Grafana, Loki, Alertmanager

Example: Simple Node.js Deployment on a VM

- Minimal steps:
  i. Provision Ubuntu VM on AWS EC2
  ii. Install Node.js and Git
  iii. Clone repo from GitHub
  iv. Run `npm ci` and `npm run start`
  v. Configure Nginx as reverse proxy
- Sample commands:

```
ssh ubuntu@EC2_PUBLIC_IP
sudo apt update && sudo apt install -y nodejs npm git
git clone https://github.com/yourname/devops-course-project.git
cd devops-course-project && npm ci && npm run start
```

Why Web Developers Must Understand Infrastructure

- “It works on my machine” is **not enough**
- Real value:
  - Apps that run **reliably** in production
- Knowledge advantages:
  - Debug production issues
  - Collaborate better with Ops/DevOps teams
  - Design systems that are easier to deploy and scale
- DevOps mindset:
  - You own the **code**, the **deployments**, and the **runtime behavior**

6. Course Overview (15-Week Roadmap)

- This course is structured as a **journey**:
  - From basic Linux/Git skills
  - To full **cloud-native DevOps pipelines**
- Each module builds on previous ones
- You should see the **narrative** clearly:
  - Manual → automated → production-ready

High-Level 15-Week Map

- Foundations:
  - Linux, Git, shell scripting
  - Basic deployment on a VM
- Cloud:
  - AWS fundamentals
- Containerization:
  - Docker, registries
- Orchestration:
  - Kubernetes, scaling
- Automation:
  - CI/CD, IaC, config management
- Observability & Security:

Example Module Flow

- Early modules:
  - **Linux, Git & Shell Scripting**
  - **SSH & Remote Server Access**
- Mid modules:
  - **Docker & Containerization**
  - **Kubernetes & Scaling**
  - **CI with GitHub Actions**
- Late modules:
  - **CD & Release Management**
  - **Terraform & Ansible**
  - **Monitoring & DevSecOps**

The Big Narrative of the Course

- Start:
  - You manually deploy a web app on one server
- Middle:
  - You containerize the app, use Docker Compose, then Kubernetes
  - You build CI pipelines to test and build images
- End:
  - You:
    - Spin up cloud infrastructure with Terraform
    - Configure servers with Ansible
    - Deploy with CI/CD
    - Monitor with Prometheus/Grafana/Loki
- Final project:

7. Course Technical Requirements

- To succeed, you must have:
  - Right **accounts**
  - Right **tools**
  - A clean **development environment**
- This section explains **why** each tool matters:
  - Not just “install this because the lecturer says so”

7.1 Required Accounts

- **GitHub**:
  - Code hosting, issues, pull requests, GitHub Actions
- **AWS Free Tier**:
  - EC2, S3, IAM, CloudWatch
- **Docker Hub**:
  - Store and share Docker images
- Optional:
  - **Grafana Cloud** for hosted dashboards
- Security:
  - Enable **2FA** on all critical accounts

7.2 Required Software (Concept)

- **VS Code**:
  - Extensions:
    - Docker, GitLens, YAML, GitHub Actions, Terraform, Ansible
- **Git & GitHub CLI**:
  - Work smoothly with GitHub from terminal
- **Languages**:
  - Node.js LTS (e.g. 18.x)
  - Java JDK (e.g. Temurin 17)
- **DevOps tools**:
  - Docker Desktop
  - AWS CLI
  - Terraform

Example: Verifying Tools

- Check versions:

```
git --version
node -v
java -version
docker --version
aws --version
terraform version
ansible --version
kubectl version --client
```

- If a command fails:
  - It is a **signal** to fix your environment early, not during an exam

7.3 Development Environment Best Practices

- Folder structure:

```
~/dev/
  502094-devops/
    notes/
    labs/
    project/
    infra/
```

- Use SSH keys instead of passwords
- Configure your shell:
  - `.bashrc`, `.zshrc`
- Keep your system:
  - Up to date
  - Backed up

SSH Keys & Secure Authentication

- Generate SSH key:

```
ssh-keygen -t ed25519 -C "student@tdtu.edu.vn"
```

- Add to ssh-agent:

```
eval "$(ssh-agent -s)"
ssh-add ~/.ssh/id_ed25519
```

- Add public key to GitHub:
  - Copy content:

    ```
    cat ~/.ssh/id_ed25519.pub
    ```

  - Paste into GitHub → Settings → SSH and GPG keys

Basic Security Hygiene for Developers

- Never commit:
  - AWS keys, DB passwords, JWT secrets, .pem files
- Use:
  - `.gitignore` for environment files
  - GitHub Secrets for CI pipelines
- Principle of **least privilege**:
  - Give accounts only the permissions they really need
- Lock your laptop and use strong passwords

Terminal Usage & Profiles

- Use a capable terminal:
  - macOS: iTerm2 / Terminal
  - Windows: Windows Terminal + WSL
- Configure:
  - Aliases (e.g. `gst` for `git status`)
  - Prompt with Git branch
- Example aliases:

```
alias ll="ls -alF"
alias gst="git status"
alias gl="git pull"
alias gp="git push"
```

8. Cloud Usage, Budgets & Safety

- Cloud is not “free” even in Free Tier
- As DevOps engineers, you must:
  - Be **cost-aware**
  - Clean up unused resources
  - Avoid surprises in billing
- This is part of **professional responsibility**

AWS Free Tier Mindset

- Free tier has:
  - limited instance hours
  - limited storage
- Guidelines:
  - Stop or terminate EC2 instances when not in use
  - Delete unused volumes, snapshots, load balancers
- Good habit:
  - End of each week:
    - Run through resources and shut down labs

AWS Budgeting & Billing Alarms

- Create a **budget alarm**:
  - Example: alert at 5 USD
- Steps:
  - In AWS console, go to **Billing → Budgets**
  - Create cost budget
  - Set email for notifications
- This is:
  - A basic form of **financial observability**

Resource Cleanup Mindset

- Before finishing a lab:
  - List resources:
    - EC2 instances, security groups, load balancers, EBS volumes
  - Decide:
    - Keep if needed soon
    - Terminate if not
- Example cleanup:

```
aws ec2 describe-instances --filters "Name=tag:Course,Values=502094"
```

- If you use Terraform:
  - Use `terraform destroy` when lab is done

Tagging Standard for This Course

- We will use tags to:
  - Identify resources created for this course
- Required tags:

```
Course = 502094
Student = <YourName>
Week   = <ModuleNumber>
```

- Example in Terraform:

```
tags = {
  Course = "502094"
  Student = "Nguyen Van A"
  Week   = "03"
}
```

9. Understanding the Final Project (High Level)

- Final project = “mini production system”
- Example scenario:
  - Backend API for a simple web application
  - Deployed to AWS with:
    - CI/CD
    - Infrastructure as Code
    - Monitoring and logging
- You will:
  - Design
  - Implement
  - Document

Final Project: DevOps Outcomes

- Expected components:
  - **Web API** (Node.js / Spring Boot)
  - **CI pipeline** (GitHub Actions)
  - **CD pipeline** (automatic deployment to EC2 or Kubernetes)
  - **Terraform** to provision infrastructure
  - **Ansible** to configure servers (if VM-based)
  - **Monitoring & logging**:
    - Prometheus + Grafana + Loki (or CloudWatch-based alternative)
  - **Security checks**:
    - Basic dependency scanning / linting

How Weekly Modules Build to Final Project

- Example mapping:
  - Week 2–3:
    - Linux + Git + server basics → you can SSH into servers
  - Week 4–6:
    - Docker + Kubernetes → you can containerize and orchestrate apps
  - Week 7–10:
    - CI/CD → you can automate builds and deployments
  - Week 11–12:
    - Terraform + Ansible → you can define and configure infrastructure
  - Week 13–14:
    - Observability + DevSecOps → you can monitor and secure your system

10. DevOps Mindset & Professional Expectations

- Technical skills are important, but:
  - **Mindset and habits** are what make you effective
- DevOps mindset:
  - Ownership
  - Automation
  - Continuous improvement
  - Collaboration

Automation Over Manual Repetition

- “If you have to do it more than twice, script it.”
- Examples:
  - Reusing commands through scripts:

```
# deploy.sh
git pull
npm ci
npm run build
pm2 restart api
```

  - Converting shell scripts into Ansible playbooks
- Automation reduces:
  - Errors
  - Cognitive load
  - Time to deploy

Infrastructure as Code Is Non-Negotiable

- Modern teams:
  - Do not “click around” in cloud consoles manually
- Instead:
  - Use tools like **Terraform** and **CloudFormation**
- Benefits:
  - Version control
  - Reproducibility
  - Easy rollback
- This course:
  - Will treat IaC as a **standard practice**, not a bonus

Observability as a First-Class Citizen

- You should always ask:
  - “How will we **know** if this is working in production?”
- Minimum for each service:
  - Meaningful logs
  - Basic metrics (latency, errors, throughput)
  - Health endpoints
- In the final project:
  - Lack of observability is a **serious weakness**

Fail Fast, Learn Quickly, Recover Cleanly

- Failure is:
  - **Expected** when experimenting in labs
- DevOps focuses on:
  - Reducing **time to detect**
  - Reducing **time to recover**
- Practices:
  - Small, reversible changes
  - Clear rollback plans
  - Post-incident reviews focused on learning, not blame

Collaboration & Documentation

- Your future team will expect:
  - Clear commit messages
  - Readable CI/CD workflows
  - Well-documented README and runbooks
- During this course:
  - Document:
    - How to set up your project
    - How to deploy
    - How to debug common issues
- Good DevOps engineers:
  - Communicate clearly and leave behind **clean trails**

Summary & Next Steps

- Today we:
  - Defined **DevOps** and its principles
  - Explored the **DevOps lifecycle** and CI/CD overview
  - Looked at the **big picture** of web deployment architectures
  - Reviewed the **course roadmap** and technical requirements
- For next session:
  - Ensure your **accounts and tools** are ready
  - Prepare to work with **Linux, Git, and shell scripting**
- Welcome to the course — let's build real DevOps skills together.